Antigravity 高性能排版技术与架构报告

基于 Headless Chromium 与现代 CSS Paged Media 的高质量 PDF 输出方案

摘要：本文档详细阐述了在 Antigravity 智能体开发环境中，如何通过 Markdown 原生语法、GitHub Flavored Callouts、KaTeX 数学公式渲染引擎以及 Headless Chromium 打印管道，实现出版级 PDF 排版与 Antigravity Artifact 双向同步预览的完整实现。

作者： Antigravity Engineering

日期： 2026-10-11

版本： v1.0.0

目录 (Table of Contents)

1. 项目概述与设计目标
2. 核心技术架构
3. 数学公式与科学计算支持
4. 代码高亮与代码块
5. 特性清单与待办任务
6. 总结与展望

1. 项目概述与设计目标

在大型语言模型与智能体开发实践中，用户经常需要将生成的复杂分析报告、调研文档或技术方案以**出版级质量**导出为 PDF。传统的纯文本或基础 Markdown 导出通常存在以下痛点：

1. **中文排版缺失**：字体粗细失真、缺失字形、换行断词不合理。
2. **公式与图表支持差**：LaTeX 公式与 Mermaid 图表在很多后端转换库中无法矢量化渲染。
3. **分页控制失控**：标题孤行落在页面底部（Orphan headings）、表格跨页被横腰截断。
4. **缺少智能体环境协同**：生成 PDF 以后无法在宿主 IDE / 对话环境中即时预览。

> **NOTE**
>
> `markdown-to-pdf` 技能专为解决上述痛点而设计。通过解耦排版模板与渲染核心，不仅支持本地毫秒级生成高质量 PDF，还能自动向 Antigravity 生成 Generative UI 预览 Artifact。

---

2. 核心技术架构

系统整体分为四个核心阶段：**Markdown 规范化与解析**、**HTML/CSS 模板注入**、**Chromium 打印引擎**以及 **Antigravity Artifact 同步**。

模块名称	核心职责	依赖技术
Parser Engine	解析 Frontmatter、转换 GitHub 警示框、提取大纲	`python-frontmatter`, `markdown`
Math & Diagram	LaTeX 数学公式矢量化、Mermaid 流程图解析	KaTeX, Mermaid.js
Style Themes	多主题排版（Modern, Academic, Report, Minimal）	CSS Paged Media (`@page`), Flexbox
Browser Runner	驱动本地 Chrome/Edge 无头打印	Headless Chromium, Virtual Time Budget
Artifact Sync	生成 Markdown 索引与 Generative UI 预览卡片	Antigravity Design System, TailwindCSS

> **TIP**
>
> 无需安装任何庞大的第三方 GTK/Pango 动态链接库。Windows 环境下直接复用系统已有的 Google Chrome 或 Microsoft Edge 浏览器，既轻量又稳定。

3. 数学公式与科学计算支持

支持行内公式如质能方程 $E = mc^2$ 以及欧拉恒等式 $e^{i\pi} + 1 = 0$。

同时支持多行独立公式块，例如高斯积分公式：

$$\int_{-\infty}^{+\infty} e^{-x^2}\,dx = \sqrt{\pi}$$

以及带有求和与分式的神经网络损失函数定义：

$$\mathcal{L}_{\text{total}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right] + \lambda \|\mathbf{W}\|_2^2$$

> 📌 **IMPORTANT**
>
> 打印引擎配备了 `--virtual-time-budget=3000` 虚拟时间预算机制，确保 KaTeX 字体与 SVG 在生成 PDF 截图前已完成全量绘制，避免公式空白或样式错位。

4. 代码高亮与代码块

针对智能体生成的代码片段，系统内置了 Pygments 语法高亮引擎，提供清晰易读的等宽字体（Cascadia Code / JetBrains Mono）支持：

```python
import datetime
from pathlib import Path
from scripts.md_to_pdf import convert_markdown_to_pdf

def build_report():
    result = convert_markdown_to_pdf(
        input_file=Path("report.md"),
        output_pdf=Path("dist/report.pdf"),
        theme="report",
        cover_page=True,
        enable_toc=True,
    )
    print(f"Generated {result['page_count']} pages, size: {result['file_size']}")

if __name__ == "__main__":
    build_report()
```

5. 特性清单与待办任务

- [x] 支持 GitHub 风格的 5 种 Callout 框体（Note, Tip, Important, Warning, Caution）
- [x] 支持 LaTeX 行内与行间数学公式排版
- [x] 支持多主题切换（现代科技、学术论文、商务研报、极简风格、GitHub风格）
- [x] 支持封面页（Cover Page）自动排版与元数据展示
- [x] 同步生成 Antigravity Generative UI Artifact 便于会话内即时交互
- [ ] 后续规划：增加自定义水印背景支持

> ⚠️ **WARNING**
> 包含较多复杂图表的长篇文档，建议在生成时保持网络畅通以加载 KaTeX 资源，或配置离线本地静态资源缓存。

---

6. 总结与展望

通过 `markdown-to-pdf` 技能，Antigravity 能够一键将回答内容升格为出版级版面的 PDF 交付物，大幅提升方案交付与报告生成的专业度与可读性。